

Robust Semi-MDP Planning via Wasserstein-1 DRO and MILP-Constrained Actor–Critic

Anonymous Submission

Abstract—We study city-scale control of electric-vehicle (EV) ride-hailing fleets where dispatch, repositioning, and charging decisions must respect charger and feeder limits under uncertain, spatially correlated demand and travel times. We formulate the problem as a hex-grid semi-Markov decision process (semi-MDP) with mixed actions—discrete (serve, reposition, charge) and continuous (charging power)—and variable action durations. To guarantee physical feasibility at both training and deployment, we learn over *intentions* produced by a masked, temperature-annealed policy. These intentions are projected, at every decision step, through a time-limited rolling mixed-integer linear program (MILP) that strictly enforces state-of-charge, port, and feeder constraints. To mitigate distributional shifts, we optimize a Soft Actor–Critic agent against a *true* Wasserstein-1 ambiguity set, employing a graph-aligned Mahalanobis ground metric to capture spatial correlations. The robust backup utilizes the Kantorovich–Rubinstein dual with a projected subgradient inner loop and a primal–dual risk budget update. Our architecture combines a two-layer GCN encoder, twin critics, and a value network driving the adversary. Experiments on a large-scale EV fleet simulator with NYC taxi data demonstrate that our method outperforms greedy heuristic and non-robust SAC baselines in net revenue while maintaining zero grid-constraint violations.

Index Terms—Electric vehicles, Fleet management, Distributionally robust optimization, Reinforcement learning, Mixed-integer linear programming.

I. INTRODUCTION

ELECTRIC-VEHICLE (EV) ride-hailing fleets are increasingly crucial for urban mobility, requiring operators to continuously assign vehicles to requests, reposition idle units, and schedule charging. These decisions must strictly adhere to state-of-charge (SoC) safety limits, charger-port capacities, and power caps, all while maintaining high service quality. However, demand and travel times vary stochastically over space and time, exhibiting correlated and nonstationary patterns. Consequently, fleet control represents a challenging constrained, sequential decision-making problem under uncertainty.

Optimization vs. Learning. Early efforts in fleet coordination primarily relied on optimization-based techniques. For instance, [1] formulated a bilevel optimization model for multimodal transit, while [2] addressed user fairness via integer programming. Similarly, [3] proposed receding-horizon control (RHC) for dispatching based on real-time sensing. While these approaches explicitly encode operational constraints, they rely heavily on accurate forecasts and suffer from scalability issues as system size increases. Conversely, Reinforcement Learning (RL) has been adopted to improve scalability and efficiency. Recent works optimize dispatching [4]–[7], vehicle repositioning [8], [9], and dynamic pricing

[10], [11] using Deep Q-Networks or Actor–Critic methods. Despite their empirical success, standard RL methods typically fail to guarantee *hard constraints* (e.g., grid limits) and remain brittle to distributional shifts between training and deployment.

The Challenge of Feasibility. Ensuring safety in RL is a longstanding challenge. Approaches like Constrained Policy Optimization [12] or Lagrangian relaxation satisfy constraints only in expectation, which is insufficient for grid-constrained operations where a single violation can trip a feeder. While differentiable optimization layers like OptNet [13] and CvxpyLayers [14] can project actions into feasible sets, they are limited to convex problems. Our domain involves discrete decisions (dispatch modes) and non-convex dynamics, necessitating a mixed-integer projection. Unlike prior works using heuristic repairs, we propose embedding a time-limited rolling MILP directly into the decision loop to guarantee physical feasibility.

Distributional Robustness. Furthermore, standard RL assumes identical training and testing dynamics. To address performance degradation under distributional shifts, robust MDPs optimize worst-case performance. Early works [15], [16] used KL-divergence or rectangular ambiguity sets, often leading to overly conservative policies. More recently, Wasserstein-based ambiguity sets have been incorporated into robust MDPs and reinforcement learning [17], [18], enabling principled robustness against distributional perturbations with provable guarantees. Foundational studies in *Distributionally Robust Optimization (DRO)* demonstrate that Wasserstein balls can effectively capture geometric perturbations in data distributions [19] and establish a close connection between DRO and adversarial training [20]. However, standard Wasserstein approaches mainly focus on unconstrained settings with continuous state spaces and typically rely on Euclidean ground metrics (L_2 distance), which ignore the underlying graph topology and fail to capture network-induced spatial interactions in transportation systems. In this work, we introduce a graph-aligned ground metric and integrate Wasserstein DRO with mixed-integer feasibility guarantees for spatially structured transportation grids.

Contributions. We develop a planning-aware, distributionally robust RL framework for controlling EV fleets on a hexagonal grid. Instead of executing actions directly, our policy outputs *intentions* that are projected through a rolling MILP. To address uncertainty, we optimize a Soft Actor–Critic (SAC) [21] agent against a Wasserstein-1 ambiguity set equipped with a graph-aligned Mahalanobis metric. Our contributions are threefold:

- (i) A semi-MDP formulation with duration-aware backups;

- (ii) A feasible-action architecture embedding a rolling MILP projection; and
- (iii) A graph-aligned Wasserstein-1 robust RL method that mitigates spatially correlated uncertainty.

II. SYSTEM MODEL AND WDRO OBJECTIVE

We consider a discrete-time horizon $t \in \{0, 1, \dots, T-1\}$ with step size $\Delta t > 0$. The service region is discretized into a hexagonal grid $\mathcal{H} = \{1, \dots, m\}$, where each hex h has a set of neighbors $\mathcal{N}(h)$. The fleet comprises N electric vehicles $\mathcal{V} = \{1, \dots, N\}$. Charging infrastructure is distributed across a subset of hexes $\mathcal{S} \subseteq \mathcal{H}$. Each station $s \in \mathcal{S}$ is characterized by:

- C_s^{port} : total number of physical charging plugs (ports) available.
- P_s^{max} : maximum charging power per vehicle at station s .
- $P_{\text{feed}}^{\text{max}}$: total power capacity of the local grid feeder supplying the station set.

Vehicle i is modeled by its current battery energy (SoC) $E_{i,t} \in [0, E_i^{\text{max}}]$, charging efficiency $\eta_c \in (0, 1]$, and energy consumption rate $\eta_{\text{drv}} > 0$ per unit distance. For $Q \succ 0$, we use the shorthand $\|x\|_Q := \sqrt{x^\top Q x}$.

A. Hex discretization and exogenous scenarios

At each step t , the system environment is driven by an exogenous scenario $\xi_t = (D_t, T_t)$. Here, $D_t \in \mathbb{R}_+^{m \times m}$ represents the Origin-Destination (OD) demand intensity, where $D_t(h_1, h_2)$ is the expected number of requests from hex h_1 to h_2 . $T_t \in \mathbb{N}^{m \times m}$ denotes the discrete travel time field in multiples of Δt , with $T_t(h_1, h_2) \geq 1$. These parameters are stochastic and spatially correlated.

B. Aggregated system state

To enable scalable learning, we adopt an aggregate state representation s_t that captures fleet dynamics at a hex-level granularity rather than tracking individual vehicles:

$$s_t = (X_t^{\text{fleet}}, X_t^{\text{demand}}, X_t^{\text{infra}}, t). \quad (1)$$

Specifically:

- $X_t^{\text{fleet}} = \{n_{h,t}^{\text{idle}}, n_{h,t}^{\text{busy}}, \bar{E}_{h,t}^{(b)}\}_{h \in \mathcal{H}}$ summarizes the counts of idle/busy vehicles and the average SoC of busy vehicles in each zone.
- $X_t^{\text{demand}} = \{D_t(h, \cdot), D_t(\cdot, h)\}_{h \in \mathcal{H}}$ summarizes the demand outflow and inflow for each hex.
- $X_t^{\text{infra}} = \{u_{s,t}, q_{s,t}, p_{s,t}^{\text{elec}}\}_{s \in \mathcal{S}} \cup \{P_{\text{feed}}^{\text{max}}\}$ captures station utilization u , queue length q , dynamic electricity price p^{elec} , and the feeder limit $P_{\text{feed}}^{\text{max}}$.

C. Scheduling objective under distributional uncertainty

Given a stationary policy π , the (episode) discounted return along a scenario sequence $\xi = (\xi_0, \dots, \xi_{T-1})$ is

$$R(\xi; \pi) = \mathbb{E} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right],$$

where r_t is the reward at decision epoch t (defined in Sec. III), and $0 < \gamma < 1$ is a discount factor.

Let $\hat{P}$ denote the empirical distribution of scenarios obtained from historical data. To hedge against distributional shifts, we optimize a distributionally robust objective over a Wasserstein-1 ball of radius $\rho > 0$ (under a metric d_Q defined later in Sec. V):

$$\max_{\pi} \inf_{P: W_1^{d_Q}(P, \hat{P}) \leq \rho} \mathbb{E}_{\xi \sim P} [R(\xi; \pi)]. \quad (2)$$

The robust backup used in our semi-MDP implementation is a sample-based approximation of (2), given in Sec. V.

III. HEX SEMI-MDP FORMULATION

Since transportation tasks (trips) and charging sessions have variable durations that may span multiple time steps Δt , we model the control problem as a Semi-Markov Decision Process (semi-MDP).

A. Per-vehicle actions and SoC Guard

For vehicle i at hex $h_{i,t}$ with SoC $E_{i,t}$, the set of valid candidate actions $\mathcal{C}_{i,t}$ includes:

$$\mathcal{C}_{i,t} = \underbrace{\mathcal{O}_{i,t}^{\text{serve}}}_{\text{orders}} \cup \underbrace{\mathcal{N}(h_{i,t})}_{\text{reposition}} \cup \underbrace{\mathcal{S}_{i,t}^{\text{chg}}}_{\text{stations}}. \quad (3)$$

However, before an order $o = (h_o^{\text{pu}} \rightarrow h_o^{\text{do}})$ is considered a valid candidate, it must pass a State-of-Charge (SoC) guard ensuring the vehicle has enough energy to reach the pickup point, complete the trip, and maintain a safety buffer E_i^{min} :

$$E_{i,t} \geq \eta_{\text{drv}} \left(e^{\text{pk}}(h_{i,t}, h_o^{\text{pu}}) + e^{\text{trip}}(h_o^{\text{pu}}, h_o^{\text{do}}) \right) + E_i^{\text{min}}. \quad (4)$$

Here $e^{\text{pk}}(h_1, h_2)$ and $e^{\text{trip}}(h_1, h_2)$ denote the energy required for pickup and trip legs, respectively. If this condition fails, the order is masked out from the vehicle's action space.

The per-vehicle action $a_{i,t}$ is a mixed tuple of discrete assignments and continuous power:

$$a_{i,t} = (\{x_{io}\}_{o \in \mathcal{O}_{i,t}^{\text{serve}}}, \{y_{ig}\}_{g \in \mathcal{N}(h_{i,t})}, \{z_{is}\}_{s \in \mathcal{S}_{i,t}^{\text{chg}}}, p_{i,t}), \quad (5)$$

subject to the constraints:

$$\begin{aligned} x_{io}, y_{ig}, z_{is} &\in \{0, 1\}, \\ \sum_o x_{io} + \sum_g y_{ig} + \sum_s z_{is} &= 1, \\ 0 \leq p_{i,t} &\leq \sum_s P_s^{\text{max}} z_{is}. \end{aligned} \quad (6)$$

Here, x, y, z are binary variables indicating the choice of serving, repositioning, or charging, respectively. The constraint $\sum x + \sum y + \sum z = 1$ ensures exactly one task is chosen per vehicle.

B. Policy parameterization (intentions)

Instead of outputting the final action $a_{i,t}$ directly (which requires satisfying complex global constraints like feeder limits), our neural policy samples *intentions* $\tilde{a}_{i,t}$. The policy factorizes into discrete mode selection and continuous power control:

$$\begin{aligned} \pi_\theta(a_{i,t}|s_t) &= \pi_\theta(m_{i,t}|s_t) \\ &\cdot \pi_\theta(\text{target}_{i,t}|m_{i,t}, s_t) \\ &\cdot \pi_\theta(p_{i,t}|m_{i,t} = \text{chg}, s_t), \end{aligned} \quad (7)$$

where $m_{i,t} \in \{\text{serve}, \text{reb}, \text{chg}\}$ is a high-level mode. These “soft” intentions are then passed to the MILP projection layer (Sec. IV) to be converted into physically feasible “hard” actions.

C. Reward

We retain exactly four components to balance revenue, costs, and service quality:

$$\begin{aligned} r_t(s_t, a_t) &= \underbrace{\sum_{o \in \text{served}_t} R_t(h_o^{\text{pu}}, h_o^{\text{do}})}_{\text{trip revenue}} - \underbrace{\sum_{i \in \mathcal{V}} c^{\text{drv}} d_{i,t}}_{\text{driving/ops cost}} \\ &\quad - \underbrace{\sum_{i \in \mathcal{V}} p_{s(i),t}^{\text{elec}} p_{i,t} \Delta t}_{\text{electricity}} \\ &\quad - \underbrace{\lambda_{\text{wait}} \sum_{o \in \mathcal{O}_t} \text{wait}_o + \lambda_{\text{drop}} |\text{dropped}_t|}_{\text{customer service.}} \end{aligned} \quad (8)$$

Here, $R_t(h_o^{\text{pu}}, h_o^{\text{do}})$ is the revenue from trip o , c^{drv} is the per-distance driving cost, $d_{i,t}$ is the distance traveled by vehicle i , $p_{s(i),t}^{\text{elec}}$ is the electricity price at station $s(i)$, wait_o is the waiting time for order o , $|\text{dropped}_t|$ is the number of dropped orders, and λ_{wait} , λ_{drop} are weighting coefficients.

D. Semi-MDP dynamics and physics

The system evolution is governed by the variable duration $\Delta(a_t)$ of the selected action:

$$\begin{aligned} \Delta_{\text{serve}} &= T_t(h_{i,t}, h_o^{\text{pu}}) + T_t(h_o^{\text{pu}}, h_o^{\text{do}}), \\ \Delta_{\text{reb}} &= T_t(h_{i,t}, g), \\ \Delta_{\text{chg}} &= 1. \end{aligned} \quad (9)$$

Charging is modeled as a 1-step action ($\Delta_{\text{chg}} = 1$) to allow frequent re-evaluation of power levels, whereas trips are atomic actions spanning multiple steps.

The physical state update equations are:

$$\begin{aligned} \text{Serve: } h_{i,t+\Delta_{\text{serve}}} &= h_o^{\text{do}}, \\ E_{i,t+\Delta_{\text{serve}}} &= E_{i,t} - \eta_{\text{drv}}(e^{\text{pk}} + e^{\text{trip}}), \\ \text{Reposition: } h_{i,t+\Delta_{\text{reb}}} &= g, \\ E_{i,t+\Delta_{\text{reb}}} &= E_{i,t} - \eta_{\text{drv}} e^{\text{rb}}(h_{i,t}, g), \\ \text{Charge: } h_{i,t+1} &= h_{i,t}, \\ E_{i,t+1} &= \min\{E_i^{\text{max}}, E_{i,t} + \eta_c p_{i,t} \Delta t\}. \end{aligned} \quad (10)$$

Here $e^{\text{rb}}(h_1, h_2)$ denotes the energy required for a reposition leg from h_1 to h_2 . Longer charging sessions are represented by repeating the charge action across multiple consecutive steps.

Aggregated idle vehicle counts $n_{h,t}^{\text{idle}}$ evolve according to a hex-level flow conservation principle balancing departures and arrivals from serving, repositioning, and completed charging; in the simulator, these counts are updated by explicitly tracking vehicles.

E. Duration-aware soft backup

In a standard MDP, future rewards are discounted by γ . In our semi-MDP, the discount factor depends on the action duration $\Delta(a_t)$, leading to the non-robust soft target

$$\begin{aligned} y_t &= r_t + \gamma^{\Delta(a_t)} \mathbb{E}_{a' \sim \pi} \left[\min_k Q_{\bar{\psi}_k}(s_{t+\Delta(a_t)}, a') \right. \\ &\quad \left. - \alpha \log \pi(a'|s_{t+\Delta(a_t)}) \right], \end{aligned} \quad (11)$$

where $(Q_{\bar{\psi}_k})$ are slowly updated target critics. This ensures that longer trips are discounted more heavily, correctly reflecting the time value of money and opportunity cost.

IV. FEASIBLE-ACTION PROJECTION VIA ROLLING MILP

Standard RL policies may output actions that violate complex coupled constraints (e.g., total power of all vehicles exceeding the grid feeder limit). To guarantee safety, we introduce a feasible-action projection layer Π_{feas} . Given the intention vector $\tilde{a}_t$ from the policy, we compute the closest feasible action a_t by solving a mixed-integer linear program (MILP).

A. Time-limited MILP formulation

Let v stack all decision variables. The MILP objective seeks to maximize revenue while staying close to the policy's intention:

$$\max_v \sum_{i,o} R_t(o) x_{io} - \sum_i c^{\text{drv}} d_{i,t} - \sum_{i,s} p_{s,i,t}^{\text{elec}} p_{i,s} \Delta t - \mu \|v - \tilde{a}_t\|_1. \quad (12)$$

This maximization is subject to critical operational constraints:

1. **Assignment logic:** Each vehicle performs exactly one task, and each order is served by at most one vehicle:

$$\sum_o x_{io} + \sum_g y_{ig} + \sum_s z_{is} = 1, \quad \sum_i x_{io} \leq 1. \quad (13)$$

2. **Infrastructure limits:** The number of charging vehicles cannot exceed port capacity C_s^{port} , and the aggregate charging power must respect the feeder limit $P_{\text{feed}}^{\text{max}}$:

$$\sum_i z_{is} \leq C_s^{\text{port}}, \quad \sum_{i,s} p_{i,s} \leq P_{\text{feed}}^{\text{max}}, \quad 0 \leq p_{i,s} \leq P_s^{\text{max}} z_{is}. \quad (14)$$

The feeder constraint couples all vehicles across stations.

3. **Positive charging:** If a vehicle chooses to charge ($z_{is} = 1$), it must draw a minimum non-zero power p_{min} to prevent “fake” charging actions:

$$p_{i,s} \geq p_{\text{min}} z_{is}. \quad (15)$$

4. **Energy feasibility:** The SoC at the next step must remain within safety bounds $[E^{\text{min}}, E^{\text{max}}]$ after accounting

Algorithm 1 Greedy Fallback Projection

```

1: Initialize empty action vector  $a_t$ 
2: Initialize state  $s_t$ , intention  $\tilde{a}_t$ , residual feeder capacity
    $P_{\text{rem}} \leftarrow P_{\text{feed}}^{\text{max}}$ 
3: Sort vehicles by increasing SoC  $E_{i,t}$ 
4: for each vehicle  $i$  do
5:   Construct feasible action set  $C_{i,t}$ 
6:   if exists feasible serving order then
7:     Select order with highest immediate reward
8:   else if exists feasible charging station with  $P_{\text{rem}} \geq$ 
    $p_{\text{min}}$  then
9:     Select nearest station  $s$  with available port
10:    Allocate power  $p_{i,s} \leftarrow \min(P_s^{\text{max}}, P_{\text{rem}})$ 
11:     $P_{\text{rem}} \leftarrow P_{\text{rem}} - p_{i,s}$ 
12:   else
13:     Select best energy-feasible reposition or idle action
14:   end if
15: end for
16: return assembled action vector  $a_t$ 

```

for consumption or charging. Let e_{io}^{od} denote the total distance for order o (pickup plus trip) if served by vehicle i , and e_{ig}^{rb} the distance for reposition from $h_{i,t}$ to g . We enforce

$$\begin{aligned}
 E_{i,t+1} &= E_{i,t} - \eta_{\text{drv}} \sum_o e_{io}^{\text{od}} x_{io} - \eta_{\text{drv}} \sum_g e_{ig}^{\text{rb}} y_{ig} \\
 &\quad + \eta_c \sum_s p_{i,s} \Delta t, \\
 E_i^{\text{min}} &\leq E_{i,t+1} \leq E_i^{\text{max}}.
 \end{aligned} \tag{16}$$

The projected total charging power is $p_i = \sum_s p_{i,s}$.

Solving this MILP at every step guarantees that the deployed action a_t is physically valid. The projection operator is defined as

$$a_t = \Pi_{\text{feas}}(s_t, \tilde{a}_t),$$

where the MILP is solved using a branch-and-bound solver under a fixed time limit τ_{max} . During the search, the solver maintains an incumbent, defined as the best feasible solution found so far. If at least one feasible solution is identified before timeout, the current incumbent is returned; otherwise, a greedy fallback rule is applied to ensure that a feasible action is always produced.

B. Greedy Fallback Projection

In rare cases where the MILP solver fails to identify any feasible solution within the time limit, no incumbent is available. To ensure that a valid action is always produced, we activate a deterministic greedy fallback procedure.

The fallback assigns decisions sequentially to vehicles in ascending order of state-of-charge (SoC), prioritizing energy-critical vehicles. For each vehicle, the algorithm selects the best feasible action subject to remaining infrastructure and energy constraints. We assume that a stay-idle action with zero energy consumption is always available in the feasible action set, which guarantees that at least one valid action exists for every vehicle at each decision step.

Specifically, the procedure operates as Algorithm 1.

C. Feasibility Guarantee

By construction, the projection operator Π_{feas} returns either (i) a MILP incumbent satisfying constraints (13)–(16), or (ii) a solution generated by the greedy fallback that explicitly enforces these constraints. Therefore, Π_{feas} always produces a physically admissible action.

V. GRAPH-ALIGNED WASSERSTEIN ROBUSTNESS

The transition dynamics of our semi-MDP depend on the exogenous scenario $\xi = (D, T)$ introduced in Sec. II. To prevent the policy from overfitting to the empirical training distribution $\hat{P}$ of such scenarios, we optimize its performance against a worst-case distribution P within a Wasserstein ambiguity set.

A. Graph-aligned Mahalanobis ground metric

Standard Wasserstein DRO uses the Euclidean distance $\|\xi - \xi'\|_2$ (e.g., [19]), which treats all scenario dimensions equally. In a city grid, perturbations should respect the spatial topology (e.g., demand shifting to a neighboring hex is more likely than shifting to a distant one). We define a graph-aligned metric d_Q :

$$d_Q(\xi, \xi') = \|Q^{1/2}(\xi - \xi')\|_2 = \|\xi - \xi'\|_Q, \tag{17}$$

where the precision matrix Q is constructed as

$$Q = \text{diag}(w) + \beta Q_{\text{graph}}.$$

Here, $\text{diag}(w)$ scales features by their inverse variance, and Q_{graph} is a Laplacian-based matrix encoding the hex grid and OD graph structure. This metric penalizes spatially disjoint perturbations more heavily than smooth, local variations.

B. Wasserstein ambiguity set

We define the ambiguity set $\mathcal{P}_\rho(\hat{P})$ as the set of all distributions P that are within a Wasserstein-1 distance ρ from the empirical distribution $\hat{P}$:

$$\mathcal{P}_\rho(\hat{P}) = \{P \in \mathcal{P}(\mathbb{R}^q) : W_1^{d_Q}(P, \hat{P}) \leq \rho\}. \tag{18}$$

C. Robust backup via Kantorovich–Rubinstein dual

Given a pre-decision state s_t and projected action $a_t = \Pi_{\text{feas}}(s_t, \tilde{a}_t)$, we denote by $s'(\xi)$ and $\Delta(\xi)$ the successor state and action duration obtained by simulating the semi-MDP dynamics in Sec. III under scenario ξ (so $s'(\hat{\xi}_t) = s_t + \Delta(a_t)$ and $\Delta(\hat{\xi}_t) = \Delta(a_t)$). The robust value target at a transition evaluates the worst-case expected value:

$$y_t^{\text{rob}} = r_t + \inf_{P \in \mathcal{P}_\rho(\hat{P})} \mathbb{E}_P[\gamma^{\Delta(\xi)} V_\varphi(s'(\xi))]. \tag{19}$$

By utilizing the identity $\inf(V) = -\sup(-V)$ and applying the Kantorovich–Rubinstein dual representation of the Wasserstein-1 ball to the cost $-V$ (see, e.g., [19], [20]), we obtain the exact dual formulation:

$$\begin{aligned}
 y_t^{\text{rob}} &= r_t + \sup_{\lambda \geq 0} \left\{ -\lambda \rho + \mathbb{E}_{\hat{P}} \left[\inf_{\xi} \left(\gamma^{\Delta(\xi)} V_\varphi(s'(\xi)) \right. \right. \right. \\
 &\quad \left. \left. \left. + \lambda d_Q(\xi, \hat{\xi}) \right) \right] \right\}.
 \end{aligned} \tag{20}$$

Our WDRO analysis assumes that the discounted value function is locally Lipschitz under d_Q . In our implementation, scenarios ξ are restricted to a compact set $\mathcal{B}$, inputs are normalized, and V_φ utilizes Lipschitz-continuous activations, ensuring this condition is practically satisfied.

In practice, we adopt a penalized, sample-based variant with a time-varying dual variable λ_t and a bounded support set $\mathcal{B}$ around each empirical sample $\hat{\xi}_t$, yielding the practical robust target:

$$y_t^{\text{rob}} = r_t - \lambda_t \rho + \min_{\xi \in \mathcal{B}} [\gamma^{\Delta(\xi)} V_\varphi(s'(\xi)) + \lambda_t d_Q(\xi, \hat{\xi}_t)]. \quad (21)$$

D. Inner minimization via projected (sub)gradient descent

To solve the inner minimization in (21), we use projected (sub)gradient descent over ξ . Let

$$f(\xi) = \gamma^{\Delta(\xi)} V_\varphi(s'(\xi)) + \lambda_t d_Q(\xi, \hat{\xi}_t).$$

We treat $\Delta(\xi)$ as piecewise constant with respect to ξ and ignore $\nabla_\xi \gamma^{\Delta(\xi)}$ in practice, leading to the update

$$\xi^{k+1/2} = \xi^k - \eta (\gamma^{\Delta(\xi^k)} \nabla_\xi V_\varphi(s'(\xi^k)) + \lambda_t g_d(\xi^k)),$$

where $\nabla_\xi V_\varphi$ is obtained via backpropagation through the value network, and g_d is the subgradient of the Wasserstein distance term:

$$g_d(\xi) = \begin{cases} \frac{Q(\xi - \hat{\xi}_t)}{\max\{\varepsilon, \|\xi - \hat{\xi}_t\|_Q\}}, & \xi \neq \hat{\xi}_t, \\ \text{any } g : \|g\|_{Q^{-1}} \leq 1, & \xi = \hat{\xi}_t. \end{cases}$$

We then project $\xi^{k+1/2}$ back to the support set $\mathcal{B}$:

$$\xi^{k+1} = \text{Proj}_{\mathcal{B}}(\xi^{k+1/2}),$$

e.g., via closed-form projection for Q -balls. Starting from $\xi^0 = \hat{\xi}_t$ and iterating K steps yields an approximate maximizer ξ^* used in (21).

E. Primal–dual risk-budget tracking

To control the *effective* adversarial radius realized by the inner maximization, we track

$$\hat{\rho}_t = d_Q(\xi_t^*, \hat{\xi}_t), \quad (22)$$

and update the dual variable λ_t toward a prescribed risk budget $\rho_{\text{target}} \in (0, \rho]$:

$$\lambda_{t+1} = \left[\lambda_t + \eta_t (\hat{\rho}_t - \rho_{\text{target}}) \right]_+, \quad \eta_t = \frac{\eta_0}{\sqrt{t}}, \quad (23)$$

where $[\cdot]_+$ denotes projection onto $\mathbb{R}_+$. This primal–dual rule penalizes the adversary when the realized radius exceeds ρ_{target} and relaxes it otherwise. Theorem 3 (Sec. VII) shows that the average budget violation vanishes as $t \rightarrow \infty$ under mild conditions.

VI. LEARNING ARCHITECTURE AND TRAINING

At each decision epoch, the simulator produces the aggregated hex-grid state s_t and scenario $\hat{\xi}_t$, as illustrated in Fig. 1. A shared hex-graph GCN encoder maps per-hex features to embeddings that are consumed by the actor, twin critics, and value network. The actor samples per-vehicle intentions, and a rolling MILP projection layer enforces SoC, port-capacity, and feeder constraints to obtain a feasible system action $a_t = \Pi_{\text{feas}}(s_t, \tilde{a}_t)$, which is executed in the semi-MDP environment. The resulting transitions are stored in a replay buffer and reused off-policy. For each sampled transition, the value network and Wasserstein adversary run the inner KR-dual loop to construct a worst-case scenario ξ^* , yielding robust targets y_t^{rob} that update the critics and the actor, while the dual variable λ is adapted to track the desired risk budget.

A. Hex-graph encoder (GCN)

We use a Graph Convolutional Network (GCN) [22] to extract spatial features. Let A be the adjacency of the hex graph (with self-loops), $\tilde{A} = A + I$, $\tilde{D} = \text{diag}(\sum_j \tilde{A}_{ij})$, and $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$. The node features Φ (vehicle counts, demand, infra features, time encodings) are processed as

$$\begin{aligned} H^{(1)} &= \sigma(\hat{A} \Phi W_0 + \mathbf{1} b_0^\top), \\ E = H^{(2)} &= \sigma(\hat{A} H^{(1)} W_1 + \mathbf{1} b_1^\top), \end{aligned} \quad (24)$$

yielding hex embeddings E that inform both the actor and critics. In our implementation, σ is the SiLU activation.

B. Actor with Gumbel–Softmax and squashed Gaussian

To allow backpropagation through the discrete action selection, we use the Gumbel–Softmax reparameterization [23]. For a set of logits (u_k) , the discrete intention $\tilde{z}$ is sampled as:

$$\begin{aligned} \tilde{z}_k &= \frac{\exp((u_k + g_k)/\tau)}{\sum_j \exp((u_j + g_j)/\tau)}, \\ g_k &\sim \text{Gumbel}(0, 1), \end{aligned} \quad (25)$$

where $\tau > 0$ is a temperature annealed during training.

For the continuous charging power $\hat{p}$, we use a squashed Gaussian head conditioned on a charging context h_i^{chg} :

$$\begin{aligned} (\mu_i, \log \sigma_i) &= \text{MLP}(h_i^{\text{chg}}), \\ u &\sim \mathcal{N}(\mu_i, \sigma_i^2), \quad \hat{p} = \frac{P_s^{\text{max}}}{2} (1 + \tanh u). \end{aligned} \quad (26)$$

The log-density includes the standard change-of-variables correction due to the tanh squashing. The actor outputs an intention $\tilde{a}_t$ built from discrete Gumbel–Softmax samples and continuous $\hat{p}$.

C. Critics and value network

Twin critics Q_{ψ_1}, Q_{ψ_2} consume a state embedding (from the GCN) and an action embedding constructed from the *projected* system action $a_t = \Pi_{\text{feas}}(s_t, \tilde{a}_t)$. A separate value network V_φ shares the GCN encoder and outputs $V_\varphi(s'(\xi))$ used by the Wasserstein adversary in (21) as well as in the soft value definition below.

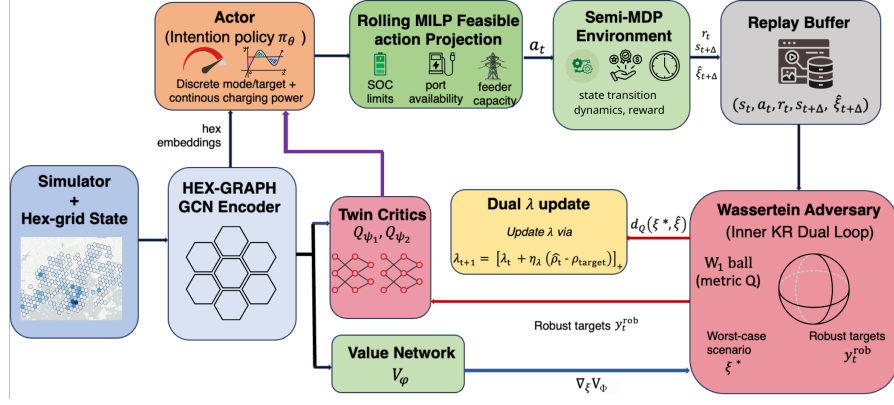


Fig. 1. Overall PD-RSAC architecture

D. Soft value and training losses

Following the SAC formulation of [21], the entropy-regularized value is defined as

$$V_\varphi(s) = \mathbb{E}_{a \sim \pi_\theta(\cdot|s)} \left[\min_{k=1,2} Q_{\psi_k}(s, \Pi_{\text{feas}}(s, a)) - \alpha \log \pi_\theta(a|s) \right]. \quad (27)$$

With the robust target y_t^{rob} from (21), we train the critics by minimizing

$$\mathcal{L}_Q(\psi_k) = \mathbb{E}[(Q_{\psi_k}(s_t, a_t) - y_t^{\text{rob}})^2], \quad k = 1, 2, \quad (28)$$

and the actor by the SAC-style [21]:

$$\mathcal{L}_\pi(\theta) = \mathbb{E}_{s, \tilde{a} \sim \pi_\theta} \left[\alpha \log \pi_\theta(\tilde{a}|s) - \min_k Q_{\psi_k}(s, \Pi_{\text{feas}}(s, \tilde{a})) \right], \quad (29)$$

where gradients through the projection Π_{feas} are handled by a straight-through estimator (STE): in the forward pass we use $a = \Pi_{\text{feas}}(s, \tilde{a})$, while in the backward pass we approximate $\partial a / \partial \tilde{a} \approx I$ along selected components. The composition of Π_{feas} with the robust backup preserves the contraction property of the Bellman operator (Theorem 1).

E. Training algorithm: PD-RSAC

The full training loop, named Primal–Dual Robust SAC (PD-RSAC), is summarized in Algorithm 2. Note that the primal–dual update (23) is already defined in Sec. V and is simply invoked here.

VII. THEORETICAL RESULTS

We briefly summarize the theoretical properties underpinning PD-RSAC.

Assumption 1 (Boundedness & discount). Rewards satisfy $|r_t| \leq R_{\max} < \infty$, and $0 < \gamma < 1$.

Assumption 2 (Feasible projection). For every state s and intention $\tilde{a}$, the projection operator $\Pi_{\text{feas}}(s, \tilde{a})$ returns a feasible action. This is guaranteed by combining the time-limited MILP (12)–(16) with a “safe” greedy fallback.

Assumption 3 (Lipschitz regularity under d_Q). On the relevant domain, $V_\varphi(s'(\cdot))$ is L_W -Lipschitz under $d_Q(\cdot, \cdot)$.

Algorithm 2 PD-RSAC with Graph-Aligned W_1 and MILP Projection

- 1: Initialize actor θ , critics ψ_1, ψ_2 , value network φ , dual $\lambda \geq 0$, replay buffer $\mathcal{D}$.
- 2: **for** each environment step t **do**
- 3: Observe s_t .
- 4: Sample intention $\tilde{a}_t \sim \pi_\theta(\cdot|s_t)$ via (7), (25), (26).
- 5: **Project:** $a_t \leftarrow \Pi_{\text{feas}}(s_t, \tilde{a}_t)$ via MILP (12)–(16).
- 6: Execute a_t in the semi-MDP, observe $r_t, s_{t+\Delta}, \hat{\xi}_t$.
- 7: Store $(s_t, \tilde{a}_t, a_t, r_t, s_{t+\Delta}, \hat{\xi}_t)$ in $\mathcal{D}$.
- 8: Sample minibatch of transitions from $\mathcal{D}$.
- 9: **for** each transition in minibatch **do**
- 10: Initialize $\xi^0 \leftarrow \hat{\xi}_t$.
- 11: **for** $k = 0$ to $K - 1$ **do**
- 12: $g \leftarrow \gamma^{\Delta(\xi^k)} \nabla_\xi V_\varphi(s'(\xi^k))$.
- 13: $h \leftarrow g_d(\xi^k)$ as in (17).
- 14: $\xi^{k+1/2} \leftarrow \xi^k - \eta(g + \lambda_t h)$.
- 15: $\xi^{k+1} \leftarrow \text{Proj}_{\mathcal{B}}(\xi^{k+1/2})$.
- 16: **end for**
- 17: $\xi^* \leftarrow \xi^K$.
- 18: Compute robust target y_t^{rob} via (21).
- 19: Update critics by minimizing (28).
- 20: Update actor by minimizing (29).
- 21: Update λ via the primal–dual rule (23).
- 22: **end for**
- 23: **end for**

A. Contraction of the robust-soft Bellman operator

For a fixed dual variable $\lambda \geq 0$, define the operator

$$(\mathcal{T}_{\text{rob}} V)(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} \left[r(s, \hat{a}) - \alpha \log \pi(\hat{a}|s) + \sup_{\xi \in \mathcal{B}} (\gamma^{\Delta(\xi)} V(s'(\xi)) - \lambda d_Q(\xi, \hat{\xi})) \right]. \quad (30)$$

with $\hat{a} = \Pi_{\text{feas}}(s, a)$ and $\Delta(\xi) \geq 1$ almost surely. The following property holds.

Theorem 1 (Contraction of robust Bellman operator). *Under Assumption 1, for any V_1, V_2 ,*

$$\|\mathcal{T}_{\text{rob}} V_1 - \mathcal{T}_{\text{rob}} V_2\|_\infty \leq \gamma \|V_1 - V_2\|_\infty.$$

Thus, $\mathcal{T}_{\text{rob}}$ is a γ -contraction in the L_∞ norm and admits a unique fixed point.

Proof. Rewards and entropy terms cancel in the difference $(\mathcal{T}_{\text{rob}}V_1 - \mathcal{T}_{\text{rob}}V_2)$. Using $\sup f - \sup g \leq \sup(f - g)$ and the fact that $\gamma^{\Delta(\xi)} \leq \gamma$ for all ξ yields

$$|(\mathcal{T}_{\text{rob}}V_1)(s) - (\mathcal{T}_{\text{rob}}V_2)(s)| \leq \gamma \|V_1 - V_2\|_\infty$$

for all s , proving the result. $\square$

B. Lipschitz robustness bound for $W_1^{d_Q}$

We recall a standard Lipschitz robustness bound for Wasserstein balls [19], specialized to d_Q .

Theorem 2 (KR Lipschitz robustness bound). *Let $g : \mathbb{R}^q \rightarrow \mathbb{R}$ be L_W -Lipschitz under d_Q , i.e., $|g(\xi) - g(\xi')| \leq L_W d_Q(\xi, \xi')$. For the ambiguity set $\mathcal{P}_\rho(\hat{P})$,*

$$\inf_{P \in \mathcal{P}_\rho(\hat{P})} \mathbb{E}_P[g] \geq \mathbb{E}_{\hat{P}}[g] - L_W \rho.$$

In particular, the worst-case expected value over $\mathcal{P}_\rho(\hat{P})$ is lower-bounded by the empirical expectation minus $L_W \rho$.

Proof. By Kantorovich–Rubinstein duality for $W_1^{d_Q}$,

$$W_1^{d_Q}(P, \hat{P}) = \sup_{\|f\|_{\text{Lip}(d_Q)} \leq 1} (\mathbb{E}_P[f] - \mathbb{E}_{\hat{P}}[f]).$$

Setting $f = g/L_W$ shows that for any P ,

$$\mathbb{E}_P[g] - \mathbb{E}_{\hat{P}}[g] \geq -L_W W_1^{d_Q}(P, \hat{P}) \geq -L_W \rho.$$

Taking the infimum over $P \in \mathcal{P}_\rho(\hat{P})$ yields the claim. $\square$

C. No-regret risk-budget tracking

Finally, we state the property of the primal–dual update (23). Let $\rho_{\max}$ denote the radius of the support set $\mathcal{B}$ under d_Q , and define $g_t := \hat{\rho}_t - \rho_{\text{target}}$, where $\hat{\rho}_t$ is given by (22). Then $|g_t| \leq \rho_{\max}$.

Theorem 3 (No-regret tracking of risk budget). *Consider the update $\lambda_{t+1} = [\lambda_t + \eta_t g_t]_+$ with $\eta_t = \eta_0/\sqrt{t}$ and g_t as above. Then the average budget violation satisfies*

$$\frac{1}{T} \sum_{t=1}^T (g_t)_+ = \mathcal{O}(T^{-1/2}),$$

so that the realized adversarial radius $\hat{\rho}_t$ tracks the desired budget ρ_{target} in an average sense.

The proof follows standard arguments for projected subgradient methods in online convex optimization ShalevShwartz2012 and is omitted for brevity.

Intuitively, Theorem 3 guarantees that the primal–dual update on λ automatically calibrates the strength of the Wasserstein penalty: when the adversary tends to exceed the desired radius ρ_{target} , the dual variable grows and pushes the realized radius back, and vice versa. As a result, the effective robustness level remains close to ρ_{target} over the course of training.

VIII. EXPERIMENTS

A. Experimental Setup

We evaluate PD–RSAC on a city-scale EV ride-hailing task built from the New York City Taxi and Limousine Commission (TLC) Trip Record Data [24]. Trips are filtered for valid coordinates and timestamps and restricted to central New York City (Manhattan and nearby boroughs). The service region is tessellated into a hexagonal grid using the H3 library [25]; at resolution 9 this yields 2811 active hexes. Each trip is mapped to an origin and destination hex by its pickup and drop-off coordinates. To match the fleet size of $N = 1000$ vehicles, we sample 20% of the daily trip data. For every $\Delta t = 5$ minutes, we aggregate trips into an OD tensor $D_t(h_1, h_2)$ and estimate a discrete travel-time field $T_t(h_1, h_2)$ from empirical median durations.

The hex adjacency used by the GCN encoder is obtained from H3 neighbor relationships. Charging stations are placed at the $|\mathcal{S}| = 150$ hexes with the largest historical pickup intensity, each with port capacity C_s^{port} and per-vehicle power cap P_s^{max} . A global feeder limit $P_{\text{feed}}^{\text{max}}$ is imposed on the total charging power. Unless otherwise stated, the fleet consists of $V = 1000$ homogeneous EVs with maximum capacity $E^{\text{max}} = 100$ kWh and initial SoCs drawn uniformly from $[0.5, 0.9]E^{\text{max}}$. Each episode simulates a full 24-hour day (288 decision steps). We split days into disjoint training, validation, and test sets by calendar date. For each method and scenario, net profit (trip revenue minus driving and electricity costs, where driving cost is \$0.30/km and electricity cost is \$0.18/kWh) as the primary performance metric.

B. Compared Methods

We evaluate four methods for EV fleet management under the same simulator, hex discretization, state representation, action space, network architecture, reward function, and training budget to ensure a fair comparison.

- **PD–RSAC (Proposed Method).** Our proposed method integrates Soft Actor-Critic with Wasserstein distributionally robust optimization and MILP-based feasible-action projection. This design combines learning-based policy optimization with optimization-based constraint enforcement to improve robustness under distributional uncertainty.
- **Greedy Heuristic.** A rule-based deterministic matching algorithm without learning. At each decision epoch, vehicles are assigned to trips by maximizing immediate profit (fare minus driving and energy costs) under a maximum pickup distance constraint. Vehicles with state-of-charge below 20% are prioritized for charging. An iterative greedy matching strategy is applied to maximize the number of served trips while satisfying energy and assignment constraints.
- **SAC (Soft Actor-Critic) Haarnoja2018.** A standard off-policy actor–critic algorithm without the MILP-based feasible-action projection or Wasserstein distributionally robust optimization components.
- **PPO (Proximal Policy Optimization) Schulman2017.** An on-policy actor–critic algorithm that stabilizes policy learning through a clipped surrogate objective. PPO jointly trains a policy (actor) and a value (critic) network, and commonly

employs generalized advantage estimation (GAE) to improve learning efficiency.

C. Main Evaluation

We evaluate PD-RSAC against five baselines on a 24-hour ride-hailing task using NYC taxi data (20% sample, $N = 1000$ vehicles, $|\mathcal{S}| = 150$ stations).

D. Ablation Study

To isolate the contributions of individual components, we conduct an ablation study by progressively removing the MILP projection layer, the Wasserstein robustness module, and the graph-aligned ground metric.

Specifically, we consider the following variants:

- **Full:** the complete PD-RSAC framework with MILP projection, Wasserstein DRO, and graph-aligned Q -metric.
- **w/o MILP:** removes the MILP-based feasibility projection while retaining the robustness module and the graph-aligned Q -metric.
- **w/o DRO:** disables the Wasserstein robustness module and trains the agent using standard SAC without distributional robustness.
- **w/o Graph:** replaces the graph-aligned Q -metric with the identity matrix, resulting in an unstructured Wasserstein ground cost.

For fair comparison, all variants are trained under identical hyperparameter settings and share the same ambiguity set radius.

E. Ablation Analysis

This section provides a qualitative interpretation of the ablation results and highlights the underlying mechanisms through which each component contributes to system performance.

Impact of MILP Projection.

Role of Distributional Robustness.

Importance of Graph-Aligned Metric.

IX. CONCLUSION

REFERENCES

- [1] K. Wei, V. Vaze, and A. Jacquillat, "Transit planning optimization under ride-hailing competition and traffic congestion," *Transportation Science*, vol. 56, no. 3, pp. 725–749, 2021.
- [2] Y. Cao, S. Wang, and J. Li, "The optimization model of ride-sharing route for ride hailing considering both system optimization and user fairness," *Sustainability*, vol. 13, no. 5, p. 2728, 2021.
- [3] F. Miao, S. Han, S. Lin, J. A. Stankovic, D. Zhang, S. Munir, H. Huang, T. He, and G. J. Pappas, "Taxi dispatch with real-time sensing data in metropolitan areas: A receding horizon control approach," *IEEE Transactions on Automation Science and Engineering*, vol. 13, no. 2, pp. 463–478, 2015.
- [4] Y. Liu, F. Wu, C. Lyu, S. Li, J. Ye, and X. Qu, "Deep dispatching: A deep reinforcement learning approach for vehicle dispatching on online ride-hailing platform," *Transportation Research Part E: Logistics and Transportation Review*, vol. 161, p. 102694, 2022.
- [5] Z. Qin, X. Tang, Y. Jiao, F. Zhang, Z. Xu, H. Zhu, and J. Ye, "Ride-hailing order dispatching at didi via reinforcement learning," *INFORMS Journal on Applied Analytics*, vol. 50, no. 5, pp. 272–286, 2020.
- [6] X. Yue, Y. Liu, F. Shi, S. Luo, C. Zhong, M. Lu, and Z. Xu, "An end-to-end reinforcement learning based approach for micro-view order-dispatching in ride-hailing," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, 2024, pp. 321–330.
- [7] J. Wang *et al.*, "Coopride: Cooperate all grids in city-scale ride-hailing dispatching with multi-agent reinforcement learning," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2025.
- [8] Y. Jiao *et al.*, "Real-world ride-hailing vehicle repositioning using deep reinforcement learning," *Transportation Research Part C: Emerging Technologies*, vol. 130, p. 103289, 2021.
- [9] J. Li and V. Allan, "Where to go: Agent guidance with deep reinforcement learning in a city-scale online ride-hailing service," in *Proceedings of the IEEE International Conference on Intelligent Transportation Systems (ITSC)*, 2022, pp. 182–189.
- [10] J. Huang *et al.*, "Deep reinforcement learning-based trajectory pricing on ride-hailing platforms," *ACM Transactions on Intelligent Systems and Technology (TIST)*, vol. 13, no. 2, pp. 1–26, 2022.
- [11] K. Jin, Z. Feng, X. Li, and F. Zhang, "Ride-hailing service pattern recognition and demand prediction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 26, 2025.
- [12] J. Achiam, D. Held, A. Tamar, and P. Abbeel, "Constrained policy optimization," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 22–31.
- [13] B. Amos and J. Z. Kolter, "Optnet: Differentiable optimization as a layer in neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 152–161.
- [14] A. Agrawal, S. Barratt, S. Boyd, and E. Busseti, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019.
- [15] G. N. Iyengar, "Robust dynamic programming," *Mathematics of Operations Research*, vol. 30, no. 2, pp. 257–280, 2005.
- [16] A. Nilim and L. El Ghaoui, "Robust control of markov decision processes with uncertain transition matrices," *Operations Research*, vol. 53, no. 5, pp. 780–798, 2005.
- [17] J. Grand-Clément and C. Kroer, "First-order methods for wasserstein distributionally robust mdps," in *Proceedings of the International Conference on Learning Representations (ICLR)*, vol. 139, 2021.
- [18] A. B. Kordabad, R. Wisniewski, and S. Gros, "Safe reinforcement learning using wasserstein distributionally robust mpc and chance constraint," *IEEE Access*, vol. 10, 2022.
- [19] P. Mohajerin Esfahani and D. Kuhn, "Data-driven distributionally robust optimization using the wasserstein metric," *Mathematical Programming*, vol. 171, no. 1, pp. 115–166, 2018.
- [20] A. Sinha, H. Namkoong, and J. Duchi, "Certifying some distributional robustness with principled adversarial training," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.
- [21] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.
- [22] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [23] E. Jang, S. Gu, and B. Poole, "Categorical reparameterization with gumbel-softmax," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [24] NYC Taxi and Limousine Commission, "Tlc trip record data," <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>, 2025, accessed: 2025-09-01.
- [25] Uber Technologies, Inc., "H3: A hexagonal hierarchical geospatial indexing system," <https://h3geo.org>, 2025, accessed: 2025-09-01.